CV Project Plan

Computer Vision Final Project — Implementation Plan

> **Deadline:** Friday, May 8, 2026 — 11:59 PM
> **Team Size:** 5 members
> **Today:** Friday, May 2 — **6 days remaining**

Project Summary

Build a **modular image processing library** from scratch and use it to solve **8 computer vision tasks**. All core algorithms must be implemented manually (no single built-in function that does the whole job). You must also collect your own datasets, write a report, and produce output images.

Project Architecture

```
final_project/
├── cv_utils/                    # Shared utility/helper module
│   ├── __init__.py
│   └── helpers.py               # Common helpers (display, save, side-by-
side compare, metrics)
│
├── tasks/                       # One script per task
│   ├── task1_selective_enhancement.py
│   ├── task2_xray_sharpening.py
│   ├── task3_auto_enhancement.py
│   ├── task4_document_cleaning.py
│   ├── task5_panorama_stitching.py
│   ├── task6_object_recognition.py
│   ├── task7_depth_approximation.py
│   └── task8_hdr_imaging.py
│
├── notebooks/                   # Jupyter notebooks for testing each task
│   ├── test_task1.ipynb
│   ├── test_task2.ipynb
│   ├── test_task3.ipynb
│   ├── test_task4.ipynb
│   ├── test_task5.ipynb
│   ├── test_task6.ipynb
│   ├── test_task7.ipynb
│   └── test_task8.ipynb
```

```
│
├── images/                      # Input images (organized by task)
│   ├── task1/ ... task8/
│
├── outputs/                     # Output images for report
│   ├── task1/ ... task8/
│
├── report/                      # Final report (PDF + source)
│   └── report.pdf
│
└── README.md
```

> **Tip**
>
> Using **Jupyter notebooks** makes it easy to show step-by-step results with inline images — great for the report too.

Technology Stack

Component	Choice
Language	Python 3.10+
OpenCV	✅ **Fully allowed** — use all `cv2` functions freely
NumPy	Array operations, masking, math
Matplotlib	Visualization, plotting, comparisons
Optional	`scikit-image` for extra utilities if needed

Task Breakdown & Approach

Task 1 — Selective Object Enhancement *(Easy)*

- `cv2.cvtColor` → HSV, create color mask with `cv2.inRange`
- `cv2.filter2D` with sharpening kernel on masked region
- `cv2.GaussianBlur` on the inverse mask
- `np.where` to composite the two results
- **Key OpenCV:** `inRange`, `GaussianBlur`, `filter2D`, `bitwise_and`

Task 2 — X-Ray Sharpening Pipelines *(Easy–Medium)*

Implement **3 different pipelines** and compare:

1. **Pipeline A:** `cv2.equalizeHist` → Unsharp mask (`cv2.GaussianBlur` + weighted add) → `cv2.medianBlur`
2. **Pipeline B:** `cv2.createCLAHE` → `cv2.Laplacian` sharpening → `cv2.medianBlur`
3. **Pipeline C:** Negative transform → Gamma correction (NumPy power) → High-boost filter (`cv2.filter2D`)

- Compare using visual side-by-side + `cv2.PSNR` or manual contrast metrics
- **Key OpenCV:** `equalizeHist` , `createCLAHE` , `Laplacian` , `GaussianBlur` , `medianBlur`

Task 3 — Auto Image Enhancement *(Medium)*

- Analyze: `cv2.meanStdDev` for brightness/contrast, `cv2.calcHist` for histogram spread
- Decision logic:
  - Dark → Gamma correction + `cv2.equalizeHist`
  - Bright → Gamma (γ > 1) + contrast stretching
  - Low contrast → `cv2.createCLAHE`
- Auto-detect and apply the best strategy
- **Key OpenCV:** `meanStdDev` , `calcHist` , `equalizeHist` , `createCLAHE`

Task 4 — Document Cleaning *(Easy–Medium)*

- `cv2.cvtColor` to grayscale
- `cv2.adaptiveThreshold` (Gaussian or Mean) for binarization
- `cv2.morphologyEx` (opening/closing) to clean noise
- Shadow removal: `cv2.GaussianBlur` (large kernel) → divide original by blurred background
- **Key OpenCV:** `adaptiveThreshold` , `morphologyEx` , `dilate` , `erode` , `GaussianBlur`

Task 5 — Panorama Stitching *(Medium–Hard)*

- `cv2.SIFT_create()` or `cv2.ORB_create()` for keypoints + descriptors
- `cv2.BFMatcher` or `cv2.FlannBasedMatcher` for feature matching + Lowe's ratio test
- `cv2.findHomography` with RANSAC
- `cv2.warpPerspective` to transform one image
- Manual blending (weighted average in overlap region) or simple paste
- **Key OpenCV:** `SIFT_create` , `BFMatcher` , `findHomography` , `warpPerspective`

Task 6 — Transformed Object Recognition *(Medium)*

- `cv2.SIFT_create()` or `cv2.ORB_create()` for invariant features
- Match template vs scene using `cv2.BFMatcher` + ratio test
- `cv2.findHomography` → verify inliers → `cv2.perspectiveTransform` for bounding box

- `cv2.polylines` to draw detected object boundary
- **Key OpenCV:** `SIFT_create`, `BFMatcher`, `findHomography`, `perspectiveTransform`

Task 7 — Depth Approximation *(Medium)*

- `cv2.cvtColor` to grayscale
- `cv2.StereoBM_create()` or `cv2.StereoSGBM_create()` for disparity computation
- Normalize disparity map to 0–255 for visualization
- Optional: `cv2.medianBlur` or `cv2.bilateralFilter` to smooth depth map
- Brighter = closer, darker = farther
- **Key OpenCV:** `StereoBM_create`, `StereoSGBM_create`, `normalize`

Task 8 — HDR Imaging *(Medium–Hard)*

- Load multiple exposures, define exposure times array
- `cv2.createCalibrateDebevec()` → estimate camera response function
- `cv2.createMergeDebevec()` → merge into HDR radiance map
- `cv2.createTonemap()` (or `cv2.createTonemapReinhard()`) → tone map to LDR
- Show individual exposures vs final HDR result
- **Key OpenCV:** `createCalibrateDebevec`, `createMergeDebevec`, `createTonemap`

Team Division

Name	Primary Tasks	Key Responsibility
Mostafa	Task 1 (Selective Enhancement) + Task 4 (Document Cleaning)	Project structure setup
Moatassem	Task 2 (X-Ray Sharpening) + Task 3 (Auto Enhancement)	Enhancement & histogram tasks (shared logic)
Refaat	Task 5 (Panorama Stitching)	Feature detection, homography & stitching
Habiba	Task 6 (Object Recognition) + Task 7 (Depth Approximation)	Feature matching + stereo vision
Adel	Task 8 (HDR Imaging) + **Report writing & compilation**	HDR pipeline + owns the final report

Why this division?

- **Mostafa** gets Tasks 1 & 4 — straightforward pipelines, plus bootstraps the project
- **Moatassem** gets Tasks 2 & 3 — share histogram/enhancement logic → efficient together

- **Refaat** gets Task 5 (panorama) — dedicated to feature matching, homography & stitching
- **Habiba** gets Tasks 6 & 7 — feature matching + stereo vision
- **Adel** gets Task 8 (HDR) — paired with report ownership since the report takes real time

---

Day-by-Day Timeline

Day	Date	Milestone
Day 1	Fri, May 2	Set up project structure. Everyone collects images for their tasks. Everyone starts their **first task**.
Day 2	Sat, May 3	Finish first tasks (Mostafa: T1, Moatassem: T2, Refaat: T5 started, Habiba: T6). Generate first output images.
Day 3	Sun, May 4	Finish first tasks. Start second tasks (Mostafa: T4, Moatassem: T3, Habiba: T7, Adel: T8). Refaat: panorama ongoing.
Day 4	Mon, May 5	All implementations should be **done or near-done**. Generate all output images.
Day 5	Tue, May 6	Code cleanup + testing. Each member writes their report sections. Adel starts compiling report.
Day 6	Wed, May 7	Report finalization, review, polish.
Day 7	Thu, May 8	Final review → **Submit before 11:59 PM**.

> **Tip**
>
> With OpenCV allowed, implementation is **much faster**. Most tasks can be done in 1 day. Use the extra time for **quality** — better images, cleaner code, stronger report.

Detailed Member Responsibilities

👨‍💻 Mostafa — Tasks 1 & 4 ⭐ (Easiest)

Day 1:

- Collect images: flowers with distinct colors → `images/task1/`, noisy scanned documents → `images/task4/`
- Download document images from: https://medium.com/data-science/denoising-noisy-documents-6807c34730c4

Day 1–2:

- **Task 1**: HSV masking → selective sharpen/blur → composite

Day 2–3:

- **Task 4**: Adaptive threshold → morphological cleanup → shadow removal

Day 5:

- Write report sections for Tasks 1 & 4

👨‍💻 Moatassem — Tasks 2 & 3

Day 1:

- Collect X-ray images (Kaggle chest X-ray dataset or similar)
- Collect dark, bright, and low-contrast images for Task 3

Day 1–3:

- **Task 2**: Build 3 sharpening pipelines, run comparisons, generate side-by-side figures

Day 3–4:

- **Task 3**: Auto-detect image type (dark/bright/low-contrast) → auto-enhance

Day 5:

- Write report sections for Tasks 2 & 3

👨‍💻 Refaat — Task 5 (Panorama)

Day 1:

- Collect overlapping photos of scenes (take them yourself or find online)

Day 1–4:

- **Task 5**: Full pipeline — SIFT/ORB → match → homography → warp → blend
- Iterate on blending quality and seam handling

Day 5:

- Write report section for Task 5 (include feature matching visualizations)

👷 Habiba — Tasks 6 & 7

Day 1:

- Collect template + scene images for object recognition (rotate/scale objects)
- Collect stereo image pairs (shift camera slightly between two shots)

Day 1–3:

- **Task 6**: SIFT/ORB matching → homography verification → draw bounding box

Day 3–4:

- **Task 7**: StereoBM/StereoSGBM disparity → depth map → smoothing

Day 5:

- Write report sections for Tasks 6 & 7

👷 Adel — Task 8 + Report Lead 📝

Day 1:

- Collect multi-exposure image sets (bracket exposures of same scene)

Day 1–3:

- **Task 8**: CalibrateDebevec → MergeDebevec → Tonemap → final HDR image

Day 3–5:

- Start report: write Introduction, Project Structure, Conclusion
- Collect all members' report sections, compile into final document

Day 6–7:

- Polish report, ensure all output images are included, proofread, handle submission

---

Report Structure

```
1. Introduction
2. Library Design & Architecture
3. Task Solutions (per task):
   a. Approach & Methodology
```

```
    b. Implementation Details
    c. Results (with output images)
    d. Observations & Discussion
4. Conclusion
5. References
```

Each member writes their own task sections (3a–3d). Adel compiles and writes sections 1, 2, 4, 5.

Verification Plan

For Each Task

- Run on **at least 3 different input images** to show robustness
- Save all intermediate and final outputs to `outputs/taskN/`
- Include before/after comparisons in the report

Automated Checks

- Ensure all scripts run end-to-end without errors
- Verify output images are generated and non-empty

Quality Checks

- Task 2: Side-by-side comparison table of 3 pipelines with metrics
- Task 3: Test on dark, bright, and low-contrast images (all 3 categories)
- Task 5: Check for visible seams in panorama
- Task 7: Verify depth map makes physical sense (near objects brighter)
- Task 8: Compare HDR result vs individual exposures

✅ Resolved

- ~~Team assignment~~ → Done (see table above)
- ~~Version control~~ → Using Git/GitHub
- ~~Project scaffolding~~ → Done (all 8 tasks + notebooks + cv_utils created)